Corectitudinea Algoritmilor

December 11, 2024

1 Introducere

O analiză utilă a unui algoritm trebuie să aibă în vedere următoarele două aspecte:

- Eficienţa (complexitatea) → cantitatea de resurse (timp şi memorie) solicitate.
- Corectitudinea → pe **orice** date de intrare corecte (i.e. care respectă un set de precondiţii) algoritmul întoarce rezultate corecte (i.e. care respectă un set de postcondiţii).

În general, problema de a demonstra că un algoritm (sau un program) are un anumit efect sau că nu are un anumit efect este nerecursivă sau nici măcar recursiv enumerabilă, după cum am văzut la începutul semestrului. În cazuri speciale, există posibilitatea de a demonstra corectitudinea unor algoritmi, care, bineînțeles, este dezirabilă pentru algoritmii pe care îi dezvoltăm. Importanța unor aplicații (de exemplu prevenirea ciocnirii de trenuri prin semnalizarea automată) și preferința pentru algoritmi demonstrabil corecți dus la dezvoltarea acestui domeniu al cunoașterii.

1.1 Demonstrarea corectitudinii [1]

Pentru a demonstra **corectitudinea**, trebuie să luăm în calcul acest "orice". Demonstraţia corectitudinii se face în **3 paşi**:

1. Identificarea precondiţiilor algoritmului. Totalitatea precondiţiilor formează **aserţiunea de intrare** notată $P_I : I \to \{0, 1\}$, unde $I =$ *domeniul datelor de intrare.*
2. Identificare postcondiţiilor algoritmului. Totalitatea postcondiţiilor formează **aserţiunea de ieşire** notată $P_O : I \times O \to \{0, 1\}$, unde $O =$ *domeniul rezultatelor*
3. Demonstraţia că, plecând din starea iniţială a algoritmului, în care se respectă aserţiunea de intrare, paşii descrişi de algoritm ne duc într-o stare finală în care se respectă aserţiunea de ieşire.

Ideea demonstraţiei este să identificăm forma generală a stărilor intermediare prin care ar trebui să treacă algoritmul şi să ne asigurăm că la fiecare acţiune algoritmul trece dintr-o stare validă în altă stare validă.
Construirea unei asemenea demonstraţii poate ajuta la identificarea şi corectarea erorilor.

1.2 Tipuri de algoritmi corecţi

- Când demonstrăm că un algoritm este *corect* şi că se termină într-un *număr finit de paşi*, spunem că algoritmul este **total corect**:

$$\forall d \in I, P_I(d) \Rightarrow \Big(termin\big(Alg(d)\big) \wedge P_O\big(d, Alg(d)\big) \Big)$$

- Când demonstrăm că un algoritm este *corect* dar doar în cazurile în care se termină într-un *număr finit de paşi*, spunem că algoritmul este **parţial corect**:

$$\forall d \in I, \Big(P_I(d) \wedge termin\big(Alg(d)\big) \Big) \Rightarrow P_O\big(d, Alg(d)\big)$$

2 Invarianţi la ciclare

2.1 Definiţie

Cea mai frecventă tehnică de demonstrare a corectitudinii algoritmilor recursivi sau iterativi este inducţia.
În cazul algoritmilor iterativi, metoda constă în identificarea şi demonstrarea de **invarianţi la ciclare**: proprietăţi valabile la intrarea într-o buclă şi conservate pe durata şi la ieşirea din buclă.

2.2 Demonstrarea invarianţilor

Invarianţii la ciclare se demonstrează în **3 paşi** care corespund, practic, unei inducţii matematice după variabila de ciclare:

1. **Iniţializare**: demonstrăm că invariantul se respectă înainte de prima iterare în buclă (iteraţia 0).
2. **Menţinere**: demonstrăm că dacă invariantul se respectă înainte de iteraţia k, atunci el se va respecta şi dupa iteraţia k (adică înainte de iteraţia k + 1).
3. **Terminare**: invariantul se respectă la ieşirea definitivă din buclă, aşa că precizăm proprietatea utilă rezultată.

2.3 Exemple

2.3.1 Suma elementelor dintr-un vector

```
1: function SUMA(V[0..n − 1])
2:     s = 0
3:     i = 0
4:     while i < n do
5:         s = s + v[i]
6:         i + +
7:     end while
8:     return s
9: end function
```

Invariant: $s_i = \sum_{j=0}^{i-1} v[j]$

1. **Iniţializare:** Înainte de prima iteraţie, $s_0 = 0 = \sum_{j=0}^{-1} v[j]$

2. **Menţinere:** Presupunem că, înainte de iteraţia i, $s_i = \sum_{j=0}^{i-1} v[j]$. După iteraţia i (înainte de iteraţia i + 1) vom avea $s_{i+1} = \sum_{j=0}^{i-1} v[j] + v[i] = \sum_{j=0}^{i} v[j]$.

3. **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $i = n$): $s_n = \sum_{j=0}^{n-1} v[j]$, adică suma tuturor elementelor din vector.

2.3.2 Sortarea prin selecţie

```
1: function SelectionSort(V[0..n − 1])
2:     for i = 0; i < n − 1; i + + do
3:         min = i
4:         for j = i + 1; j < n; j + + do
5:             if v[j] < v[min] then
6:                 min = j
7:             end if
8:         end for
9:         swap(v[i], v[min])
10:    end for
11:    return V
12: end function
```

1. **Invariant pentru bucla interioară:** Înainte de iteraţia j:
   min_j = indexul elementului minim din $v[i..j-1]$

   (a) **Iniţializare:** Înainte de prima iteraţie, $min_{i+1} = i$ = indexul elementului minim din $v[i..i]$

   (b) **Menţinere:** Presupunem că, înainte de iteraţia j,
   min_j = indexul elementului minim din $v[i..j-1]$
   După iteraţia j (înainte de iteraţia j + 1), dacă $v[j] < v[min_j]$, atunci $min_{j+1} = j$, altfel $min_{j+1} = min_j$.
   Aşadar, min_{j+1} = indexul elementului minim din $v[i..j]$

   (c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $j = n$):
   min_n = indexul elementului minim din $v[i..n-1]$

2. **Invariant pentru bucla exterioară:** Înainte de iteraţia i, $v[0..i-1]$ este sortat **şi conţine cele mai mici i elemente din v**.

   (a) **Iniţializare:** Înainte de prima iteraţie, $v[0..-1]$ este sortat şi conţine cele mai mici 0 elemente din v.

   (b) **Menţinere:** Presupunem că, înainte de iteraţia i, $v[0..i-1]$ este sortat şi conţine cele mai mici i elemente din v.
   După iteraţia i (înainte de iteraţia i + 1), $v[0..i-1]$ rămâne nemodificat, iar $v[i]$ primeşte valoarea $v[min]$, unde min = indexul elementului minim din $v[i..n-1]$ (conform invariantului anterior).

Aşadar, înainte de iteraţia i + 1, $v[0..i]$ sortat şi conţine cele mai mici i + 1 elemente din v.

(c) **Terminare:** La ieşirea definitivă din buclă (înainte de iteraţia cu $i = n - 1$): $v[0..n-2]$ este sortat şi conţine cele mai mici n - 1 elemente din v. Rezultă că $v[n-1]$ conţine cel mai mare element din v, deci $v[0..n-1]$ sortat.

Prin invarianții selectați am demonstrat că funcția SelectionSort sortează vectorul V.

3 Exerciţii

3.1 Căutarea binară [2]

```
1: function BinarySearch(V, x, start, stop)
2:     while start ≤ stop do
3:         mid = (start + stop)/2
4:         if (x == v[mid]) then return mid
5:         end if
6:         if (x > v[mid]) then
7:             start = mid + 1
8:         else
9:             stop = mid − 1
10:        end if
11:    end while
12:    return Null
13: end function
```

3.2 Sortarea prin inserţie

```
1: function InsertionSort(V[0..n − 1])
2:     for j = 1; j < n; j + + do
3:         x = v[j]
4:         i = j − 1
5:         while i >= 0 and v[i] > x do
6:             v[i + 1] = v[i]
7:             i − −
8:         end while
9:         v[i + 1] = x
10:    end for
11: end function
```

3.3 Sortarea prin metoda bulelor

```
1: function BubbleSort(V[0..n − 1])
2:     for i = 0; j < n; j + + do
3:         for j = n − 1; j > i; j − − do
4:             if v[j] < v[j − 1] then
5:                 swap(v[j], v[j − 1])
6:             end if
7:         end for
8:     end for
9: end function
```

3.4 Funcţia de partiţionare din cadrul algoritmului QuickSort [3]

```
1: function Partition(V, start, stop)
2:     x = v[stop]
3:     i = start − 1
4:     for j = start; j < stop; + + j do
5:         if v[j] < x then
6:             + + i
7:             swap(v[i], v[j])
8:         end if
9:     end for
10:    swap(v[i + 1], v[stop])
11:    return i + 1
12: end function
```

3.5 Funcţia de interclasare din cadrul algoritmului MergeSort

```
1: function MERGE(A, p, q, r)
2:     n1 = q − p + 1
3:     n2 = r − q
4:     for i = 1; i <= n1; + + i do
5:         L[i] = A[p + i − 1]
6:     end for
7:     L[n1 + 1] = ∞
8:     for i = 1; i <= n2; + + i do
9:         R[i] = A[q + i]
10:    end for
11:    R[n2 + 1] = ∞
12:    i = 1
13:    j = 1
14:    for k = p; k ≤ r; + + k do
15:        if L[i] <= R[j] then
16:            A[k] = L[i]
17:            i = i + 1
18:        else
19:            A[k] = R[j]
20:            j = j + 1
21:        end if
22:    end for
23: end function
```

3.6 Determinarea distanţei minime prin algoritmul lui Dijkstra

```
1: function DIJKSTRA(G(V, E, cost), source)
2:     PQ = PriorityQueue()
3:     d = [∞ for v ∈ V]
4:     d[source] = 0
5:     for each v ∈ V do
6:         PQ.insert(v, d[v])
7:     end for
8:     while not PQ.empty() do
9:         w = PQ.getMin()
10:        PQ.removeMin()
11:        for each v ∈ G.adj[w] do
12:            if d[w] + cost(w, v) < d[v] then
13:                d[v] = d[w] + cost(w, v)
14:                PQ.update(v, d[v])
15:            end if
16:        end for
17:    end while
18: end function
```

References

[1] C. A. Giumale, *Introducere în Analiza Algoritmilor*. Polirom, 2004.

[2] "Invariants for binary search." Available at `http://www.cs.cornell.edu/courses/cs2112/2015fa/lectures/lec_loopinv/index.html` Accessed 10.11.2019.

[3] "Invariants for quicksort." Available at `http://www.columbia.edu/~cs2035/courses/csor4231.F03/quicksort-notes.pdf` Accessed 10.11.2019.